

# A Compact FPGA-Based AWGN Generator Using the Table-Driven Box–Muller Method with Central-Limit Smoothing: Bit-Exact Co-Design and Verification

Nguyen Pham Tuan<sup>1</sup>

Class IC2007 – Student ID SE201637,  
IC Design, FPT University, Ho Chi Minh City, Vietnam  
nguyenphamtuan67@gmail.com

**Abstract.** Hardware-based channel emulation requires additive white Gaussian noise (AWGN) sources that combine high statistical accuracy with high throughput and a small logic footprint. This paper presents the design, fixed-point modeling, and bit-exact verification of a compact AWGN generator targeting FPGA implementation. The generator follows the table-driven Box–Muller architecture of Danger et al., in which the two nonlinear functions  $\sqrt{-\ln x}$  and  $\sqrt{2}\cos(2\pi x)$  are replaced by two small look-up tables of  $80 \times 9$  and  $256 \times 9$  bits, respectively; a logarithmic segmentation of the uniform variable, obtained directly from the leading-zero count of a 32-bit combined Tausworthe word, preserves the accuracy of the distribution tail. A central-limit accumulation of four consecutive samples smooths the residual quantization error at the cost of a single arithmetic shift. The design is developed through a three-tier co-design flow—a floating-point reference, a bit-accurate fixed-point model, and synthesizable Verilog RTL—in which the fixed-point model generates golden vectors that the RTL reproduces with zero mismatches over 18,000 compared samples, including a stall test that exercises pipeline back-pressure. With an output resolution of 6 fractional bits, the generated noise attains a standard deviation of 1.008, an excess-free kurtosis of 2.975, and a Kolmogorov–Smirnov distance of 0.49 %; bit-error-rate simulations of BPSK and QPSK over 0–9 dB  $E_b/N_0$  match theory with an average ratio of 0.97. The generator outputs one sample every four clock cycles, corresponding to 12.5 MS/s at a 50 MHz clock, with an estimated cost of roughly 280 LUTs and one  $9 \times 9$  multiplier.

**Keywords:** AWGN · Gaussian noise generator · Box–Muller · FPGA · Fixed-point arithmetic · Bit-exact verification · Channel emulation.

## 1 Introduction

Assessing the bit-error-rate (BER) behavior of a digital transceiver at operating points around  $10^{-6}$  requires on the order of  $10^7$  to  $10^8$  noise samples per measured point. Software generation of such sample volumes is too slow to be

practical when the device under test is itself implemented in hardware; the established solution is to embed an AWGN generator next to the datapath under test, so that noise is produced in real time on the same FPGA [6, 10, 11, 4]. The AWGN channel is the canonical model of information theory [13], and it remains the baseline over which modern systems are evaluated, including learning-based end-to-end transceivers [2], which further motivates fast and accurate hardware noise sources.

A hardware Gaussian noise source must satisfy three requirements simultaneously. First, the distribution must be accurate not only near the mean but far into the tails, because the errors that determine a BER of  $10^{-6}$  are caused by noise excursions of roughly  $4.7\sigma$ ; a generator that is only accurate to  $\pm 2\sigma$  produces optimistic BER curves. Second, the throughput must reach millions of samples per second so that a BER point can be measured in seconds. Third, the logic and memory cost must remain small, since the generator shares the FPGA with the system under test.

Among the algorithmic families surveyed in [11], the Box–Muller transformation [1] is attractive for hardware because it produces exactly Gaussian samples from uniform inputs, with tail accuracy limited only by the numeric representation of its two nonlinear functions. Direct evaluation of  $\sqrt{-\ln x}$  and of the trigonometric factor is, however, expensive in logic. This paper adopts the table-driven formulation introduced by Danger, Ghazel, Boutillon, and Laamari [6], which reduces both functions to small ROMs by exploiting a logarithmic segmentation of the uniform input, and combines it with a light central-limit stage.

The contribution of this work is twofold. First, a complete and internally consistent fixed-point parameterization of the architecture is derived and validated, including the word lengths of both look-up tables, the rounding scheme of the multiplier output, and the sign-mapping convention of [6]. Second, a three-tier co-design and verification flow is described in which a bit-accurate software model generates golden vectors that the synthesizable RTL reproduces exactly, sample by sample, so that every statistical result measured on the software model is valid, by construction, for the hardware.

The remainder of the paper is organized as follows. Section 2 summarizes the algorithm and its fixed-point parameter set. Section 3 describes the hardware architecture. Section 4 presents the verification methodology. Section 5 reports statistical, BER, and word-length results, and Sect. 6 concludes.

## 2 Algorithm and Fixed-Point Parameterization

### 2.1 Box–Muller Transformation

Given two independent uniform variables  $x_1, x_2 \in (0, 1)$ , the Box–Muller transformation [1] produces a standard normal sample

$$n = \sqrt{-\ln x_1} \cdot \sqrt{2} \cos(2\pi x_2) = f(x_1) \cdot g(x_2), \quad (1)$$

where the factor  $\sqrt{2}$  is absorbed into the trigonometric term  $g(\cdot)$ , so that  $f(x) = \sqrt{-\ln x}$ . This split matches the convention of [6] and determines the contents of the two look-up tables described below.

## 2.2 Uniform Source: Combined Tausworthe Generator

The uniform bits are produced by the three-component combined Tausworthe generator of L'Ecuyer [9], which delivers one maximally equidistributed 32-bit word per clock cycle with a period of approximately  $2^{88}$ . The generator costs only a few shift and XOR operations per component and has no multiplier, which makes it well suited to FPGA implementation. The seeds must respect the component constraints  $s_1 > 1$ ,  $s_2 > 7$ ,  $s_3 > 15$ ; the values (12345, 67891, 11213) are used throughout this work.

## 2.3 Logarithmic Segmentation and Table Contents

The function  $f(x) = \sqrt{-\ln x}$  varies slowly near  $x = 1$  but steeply as  $x \rightarrow 0$ , which is precisely the region that generates the distribution tail. A uniform segmentation of  $(0, 1)$  would therefore require a very large table. Following [6], the interval is instead segmented in powers of 16: the number of leading zeros  $lz$  of the 32-bit uniform word selects a segment index

$$r = \min(\lfloor lz/4 \rfloor + 1, K), \quad K = 5, \quad (2)$$

so that segment  $r$  corresponds to  $x \in [16^{-r}, 16^{-(r-1)})$ , an event of probability  $16^{-(r-1)} \cdot \frac{15}{16}$ . The four bits that follow the leading-zero prefix select one of 16 subsegments  $s$  inside the segment. The input word thus falls into a subsegment with exactly the probability of the corresponding  $x$ -interval, and no division or comparison is needed—only a leading-zero count. The first table stores

$$F_r(s) = \text{round}(2^m \sqrt{-\ln \tilde{x}_{r,s}}), \quad m = 7, \quad (3)$$

where  $\tilde{x}_{r,s}$  is the representative point of subsegment  $(r, s)$ . With  $K = 5$  segments of 16 entries, the table has 80 entries; the maximum value is 470, which fits an unsigned 9-bit word (format Q2.7).

The second factor of (1) is a smooth bounded function and is tabulated uniformly: the eight least-significant bits of the uniform word address a 256-entry table

$$G(s') = \text{round}\left(2^{m'} \sqrt{2} \cos(\pi(s' + \frac{1}{2})/512)\right), \quad m' = 7. \quad (4)$$

The maximum magnitude is  $|G|_{\max} = \text{round}(2^7 \sqrt{2}) = 181 > 127$ , so a signed representation requires 9 bits (format Q1.7). One further bit of the uniform word carries the sign; following Eq. (7) of [6], the signed output is formed as  $P$  or  $-P - 1$ , which offsets the two half-distributions by exactly one LSB and is compensated by a constant in the accumulation stage.

**Table 1.** Fixed-point parameter set of the implemented generator.

| Parameter | Value | Meaning                                                    |
|-----------|-------|------------------------------------------------------------|
| $K$       | 5     | number of logarithmic segments of $f$                      |
| $m$       | 7     | fractional bits of table $F_r$ (Q2.7, unsigned, 9 bits)    |
| $m'$      | 7     | fractional bits of table $G$ (Q1.7, signed, 9 bits)        |
| $B$       | 6     | fractional bits of the output sample                       |
| $A$       | 4     | samples accumulated by the central-limit stage             |
| Output    | Q2.6  | signed 16-bit container                                    |
| URNG      | —     | combined Tausworthe, 3 components, period $\approx 2^{88}$ |

## 2.4 Central-Limit Smoothing

A single table-driven Box–Muller sample still carries quantization error. A final stage accumulates  $A = 4$  consecutive samples and rescales by  $\sqrt{A} = 2$ :

$$y = \left( \sum_{i=1}^4 b_i + \frac{A}{2} \right) / 2, \quad (5)$$

where the constant  $A/2$  compensates the  $-\frac{1}{2}$  LSB bias introduced by the sign mapping, and the division is an arithmetic right shift. The central-limit stage smooths the discrete levels of the quantized samples and improves the match to the normal law, at the cost of dividing the output rate by four. Table 1 summarizes the complete parameter set.

## 3 Hardware Architecture

The RTL consists of seven small Verilog modules organized in four functional blocks.

**Uniform Source.** `taus_urng` implements the three Tausworthe components and delivers one 32-bit word per enabled clock cycle together with a `valid` strobe that is asserted only in cycles in which the output word has been updated, so that downstream stages consume each word exactly once even when the pipeline is stalled.

**Bit Extraction.** `lzc32` computes the leading-zero count with a five-level binary-search structure (logic depth  $\mathcal{O}(\log_2 32)$  instead of a 32-stage priority chain), from which the indices  $r$ ,  $s$ , the table address  $s'$ , and the sign bit are derived by simple wiring and a barrel selection.

**Box–Muller Core.** `fr_rom` ( $80 \times 9$ , unsigned) and `g_rom` ( $256 \times 9$ , signed) are read combinatorially. `bm_core` multiplies the two table outputs ( $9 \times 9 \rightarrow 19$  bits, one DSP or LUT-based multiplier), applies round-half-up rescaling by

$2^{m+m'-B} = 2^8$  as  $(P + 128) \gg 8$  with an arithmetic shift—which is bit-identical to the  $\lfloor (P + 128)/256 \rfloor$  operation of the software model for signed  $P$ —applies the sign mapping  $P \mapsto -P - 1$ , and saturates to a signed 16-bit word. An optional parameter PIPE inserts one register stage between the table look-up and the multiplier, halving the critical path at the cost of one cycle of latency; the output values are unaffected.

**Central-Limit Accumulator.** `clt_acc` accumulates four valid samples, adds the compensation constant  $A/2 = 2$ , performs the arithmetic right shift by one, saturates, and asserts an output strobe. The top level `awgn_top` parameterizes the seeds, all word lengths, and PIPE.

The generator therefore produces one output sample every four clock cycles. At a 50 MHz clock this corresponds to 12.5 MS/s. An analytical estimate of the resource cost is approximately 280 LUTs, 150 flip-flops, one  $9 \times 9$  multiplier, and  $80 \times 9 + 256 \times 9 = 3024$  ROM bits, which synthesis tools typically map to distributed memory.

## 4 Verification Methodology

### 4.1 Three-Tier Co-Design Flow

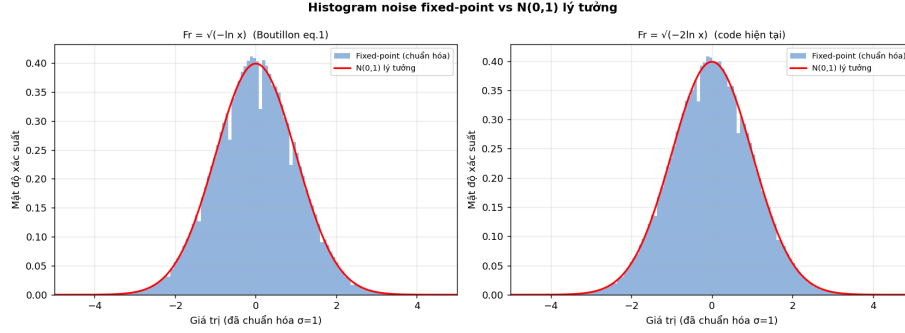
The design is developed against two software models. A *floating-point reference* evaluates (1) in double precision and is used to validate the algorithmic choices—the segmentation, the table generation formulas, and the central-limit stage—by measuring the first four moments of the generated distribution. A *bit-accurate fixed-point model* then reproduces every hardware operation using integer arithmetic only: table quantization by rounding, integer multiplication, the  $\lfloor (P + 128)/256 \rfloor$  rounding step, the  $-P - 1$  sign mapping, and the final accumulation, shift, and saturation. Because the fixed-point model and the RTL implement the same integer operations, their outputs can be required to agree *exactly*, bit for bit. During model development, publicly available open-source implementations of AWGN generation and AWGN channel simulation in C++, Python, and MATLAB [7, 12, 8, 14, 3], as well as an HDL generator project [5], were consulted as additional cross-references for algorithmic conventions such as noise-power scaling and seeding.

### 4.2 Golden Vectors and Testbenches

The fixed-point model exports the two table images and three golden sequences generated from the reference seeds: 8000 URNG words, 8000 Box–Muller samples, and 2000 output samples. Four self-checking testbenches compare the RTL against these vectors with exact (`!=="`) comparison: (i) the URNG in isolation; (ii) the Box–Muller datapath driven by recorded URNG words; (iii) the complete self-running top level; and (iv) a stall test in which the enable input is toggled in bursts, verifying that the output stream is invariant under pipeline

**Table 2.** Sample statistics of the fixed-point output ( $N = 3 \times 10^5$ ,  $B = 6$ ).

|          | Mean   | Std. dev. | Skewness | Kurtosis | KS distance |
|----------|--------|-----------|----------|----------|-------------|
| Measured | +0.004 | 1.008     | 0.006    | 2.975    | 0.49 %      |
| Ideal    | 0      | 1         | 0        | 3        | 0           |

**Fig. 1.** Normalized histogram of the fixed-point output versus the  $\mathcal{N}(0,1)$  density.

back-pressure. In addition, a cycle-accurate software co-simulator replays the RTL register-transfer semantics and checks all six configurations  $\{\text{PIPE} = 0, 1\} \times \{\text{continuous, bursty, random enable}\}$  against the same golden vectors.

Across all tests, the RTL reproduces the golden vectors with **zero mismatches** over the 18,000 compared samples, and the six co-simulated configurations pass without deviation. Consequently, all statistical results reported in Sect. 5, although measured on the fixed-point model for speed, hold exactly for the hardware.

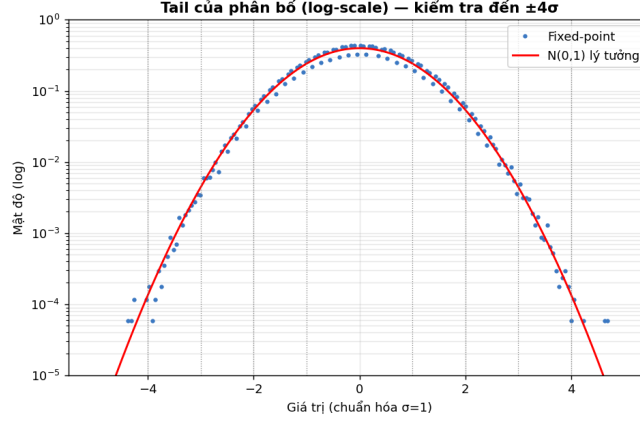
## 5 Results

### 5.1 Statistical Quality

Table 2 lists the sample moments of  $3 \times 10^5$  output samples together with the Kolmogorov–Smirnov (KS) distance to the standard normal distribution. The kurtosis of 2.975 is within 0.9 % of the Gaussian value of 3, and the KS distance is below 0.5 %. Figure 1 compares the normalized histogram with the  $\mathcal{N}(0,1)$  density, and Fig. 2 shows the same comparison on a logarithmic scale: the empirical tail follows the normal law out to  $\pm 4\sigma$ , i.e., down to probability densities of order  $10^{-4}$ – $10^{-5}$ , which covers the noise excursions that dominate BER measurements at  $10^{-6}$ .

### 5.2 Bit-Error-Rate Validation

The generator was used as the noise source of a baseband BER simulation of BPSK and QPSK over  $E_b/N_0 = 0$ –9 dB. Figure 3 shows the BPSK result: the



**Fig. 2.** Distribution tail on a logarithmic scale. The empirical density follows the normal law out to  $\pm 4\sigma$ .

**Table 3.** Effect of the output fractional word length  $B$  ( $N = 3 \times 10^5$ ).

| $B$ | Bulk error | KS distance | Distinct output levels |
|-----|------------|-------------|------------------------|
| 4   | 8.15 %     | 1.74 %      | 134                    |
| 6   | 3.42 %     | 0.47 %      | 495                    |
| 8   | 2.61 %     | 0.18 %      | 1818                   |
| 10  | 2.52 %     | 0.16 %      | 6363                   |

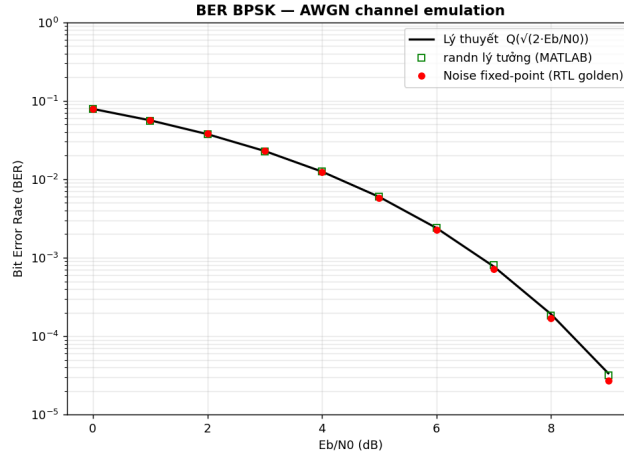
curve obtained with the fixed-point noise is indistinguishable from the analytical reference  $Q(\sqrt{2E_b/N_0})$  down to BER values of a few  $10^{-5}$ , the resolution limit of the simulated sample count. Over both modulations and the full  $E_b/N_0$  range, the average ratio between the measured and theoretical BER is 0.97.

### 5.3 Output Word-Length Study

Table 3 reports the effect of the output fractional word length  $B$  on the distribution quality. With  $B = 4$  the output is visibly staircase-quantized (bulk error above 8 %). Increasing  $B$  beyond 6 enlarges every downstream datapath while improving the bulk error by less than one percentage point. The value  $B = 6$  is therefore retained: it keeps the KS distance below 0.5 % and preserves the tail accuracy required at  $\pm 4\text{--}5\sigma$ , at minimum hardware cost.

### 5.4 Throughput and Cost

With  $A = 4$ , the generator delivers one sample every four cycles; at 50 MHz this is 12.5 MS/s, sufficient to accumulate the  $10^7\text{--}10^8$  samples of a  $10^{-6}$  BER point in seconds. The estimated cost—about 280 LUTs, 150 flip-flops, one  $9 \times 9$  multiplier,



**Fig. 3.** BPSK bit-error rate using the fixed-point noise versus the analytical reference.

and 3024 ROM bits—leaves essentially the whole device to the system under test. If one sample per cycle is required, four Box–Muller cores can be instantiated in parallel and summed directly, since the central-limit stage is associative; the optional PIPE stage supports higher clock targets without changing any output value.

## 6 Conclusion

This paper presented a compact FPGA AWGN generator based on the table-driven Box–Muller architecture of [6] with a four-sample central-limit stage, together with a complete fixed-point parameterization ( $K = 5$ ,  $m = m' = 7$ ,  $B = 6$ ,  $A = 4$ ) and a three-tier verification flow. The synthesizable RTL reproduces the bit-accurate software model with zero mismatches over 18,000 samples, including under pipeline stalls, so the measured statistics—standard deviation 1.008, kurtosis 2.975, KS distance 0.49 %, and BER curves matching theory with an average ratio of 0.97—are guaranteed to hold for the hardware. Future work includes reporting post-synthesis resource and timing figures on a concrete device and instantiating parallel Box–Muller cores for one-sample-per-cycle operation.

## References

1. Box, G.E.P., Muller, M.E.: A note on the generation of random normal deviates. *The Annals of Mathematical Statistics* **29**(2), 610–611 (1958)
2. Chen, J., Zhang, L., Chen, Z., Chen, Y., Zhuang, H.: GAN-based generator of adversarial attack on intelligent end-to-end autoencoder-based communication system. *arXiv:2505.00395* (2025)



3. chh-han: white\_noise\_generation: White noise generation in python. [https://github.com/chh-han/white\\_noise\\_generation](https://github.com/chh-han/white_noise_generation), GitHub repository, accessed 2026
4. Cortez, A., Franco, Z., Borjas, J.: Noise generator by free FPGA technology. *Universidad, Ciencia y Tecnologia* **27**(121), 64–73 (2023). <https://doi.org/10.47460/uct.v27i121.755>
5. crboth: AWGN\_Generator: An HDL AWGN generator for FPGA. [https://github.com/crboth/AWGN\\_Generator](https://github.com/crboth/AWGN_Generator), GitHub repository, accessed 2026
6. Danger, J.L., Ghazel, A., Boutillon, E., Laamari, H.: Efficient FPGA implementation of Gaussian noise generator for communication channel emulation. In: *Proc. 7th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*. pp. 366–369. Jounieh, Lebanon (2000)
7. fualsan: A.W.G.N.: Additive white gaussian noise in C++. <https://github.com/fualsan/A.W.G.N.>, GitHub repository, accessed 2026
8. igorauad: awgn\_matlab: AWGN channel simulation in MATLAB. [https://github.com/igorauad/awgn\\_matlab](https://github.com/igorauad/awgn_matlab), GitHub repository, accessed 2026
9. L'Ecuyer, P.: Maximally equidistributed combined Tausworthe generators. *Mathematics of Computation* **65**(213), 203–213 (1996)
10. Lee, D.U., Villasenor, J.D., Luk, W., Leong, P.H.W.: A hardware Gaussian noise generator using the Box–Muller method and its error analysis. *IEEE Transactions on Computers* **55**(6), 659–671 (2006). <https://doi.org/10.1109/TC.2006.81>
11. Malik, J.S., Hemani, A.: Gaussian random number generation: A survey on hardware architectures. *ACM Computing Surveys* **49**(3), 53:1–53:37 (2016). <https://doi.org/10.1145/2980052>
12. mskwarek: AWGN: Python implementation of an AWGN channel. <https://github.com/mskwarek/AWGN>, GitHub repository, accessed 2026
13. Tridenski, S., Somekh-Baruch, A.: The method of types for the AWGN channel. *arXiv:2307.13322 [cs.IT]* (2023)
14. zhanxn87: awgn\_boxmuller: Box–Muller based AWGN generation. [https://github.com/zhanxn87/awgn\\_boxmuller](https://github.com/zhanxn87/awgn_boxmuller), GitHub repository, accessed 2026